Alliance University

Alliance College of Engineering and Design

Micro-project Report

MATLAB-Based Public-Key Cryptography and Secure Communication Using Group Theory and Modular Arithmetic

Sets, Relations and Functions · Graph Theory · Group Theory · Diffie–Hellman Key Exchange · RSA

Team members

Santanu Barua · M Hariharan · Kushi Pathak

P Abdul Wasim · Someshwar A · Adithya Sampath Kumar

Contents

Abstract 3

1 Introduction 4

1.1 Problem and motivation 4

1.2 Objectives 4

1.3 Scope and what is not claimed 4

1.4 Organisation of the report 4

2 Mathematical background 5

2.1 Sets, relations and functions 5

2.2 Graph theory 5

2.3 Modular arithmetic 6

2.4 Group theory 6

2.5 The Diffie–Hellman key exchange 6

2.6 The RSA cryptosystem 7

3 System design 8

3.1 The users and the network 8

3.2 Architecture of the modules 8

4 Implementation 10

4.1 Code organisation and style 10

4.2 The cipher with the key (module sets_relations_functions) 10

4.3 The graph (module graph_network) 10

4.4 Modular arithmetic and the group (module group_theory) 10

4.5 Diffie–Hellman and encryption (modules diffie_hellman, encryption) 11

4.6 The secure network and the routing (module integration) 11

4.7 The eavesdroppers 12

4.8 RSA (module rsa) 12

5 Results and verification 14

5.1 The test suite 14

5.2 The Diffie–Hellman key exchange 14

5.3 Encryption with the shared key 15

5.4 The network at work 16

5.5 Verification by a second method 16

5.6 RSA results 18

5.7 Figures of the network 18

6 Security analysis and limitations 21

7 Conclusion and future work 22

References 22

Appendix A: How to run the project 23

Appendix B: The code, file by file 24

Abstract

This project builds a small but complete simulation of secure communication in MATLAB. A network of six users (Alice, Bob, Carol, Dave, Frank and Grace) is joined by six communication channels. Every channel is protected by its own secret key, and that key is never sent: the two users at the ends of the channel create it with the Diffie–Hellman key exchange in the cyclic group Z_{97}^{ι}. The key then drives a cipher, and a message travels hop by hop along the shortest path found by breadth-first search. RSA public-key encryption is added as a second use of the same mathematics, including the transport of a cipher key under an RSA public key.

Four areas of discrete mathematics are used as working tools and not only as theory: sets, relations and functions model users, symbols, keys and the cipher; graph theory models and analyses the network; group theory and modular arithmetic give the key exchange and RSA; and number theory (Euclid’s algorithm, fast exponentiation, Lagrange’s theorem) supplies the algorithms. The code has 116 files (about 4,650 lines of base MATLAB with no toolboxes), a test suite of 1,094 automatic checks in 14 test files that all pass in GNU Octave 8.4, and a verification layer that checks every important result a second way with code that shares nothing with the part it checks. Eavesdroppers are simulated on purpose: they break the small examples, which shows why real systems need large numbers. The project is an educational model and not a security product.

Keywords: Diffie–Hellman, RSA, cyclic group, modular arithmetic, graph theory, secure communication, MATLAB.

1 Introduction

1.1 Problem and motivation

Two people who have never met cannot simply agree on a secret key if every message between them can be read by others. This is the key-distribution problem, and the answer of public-key cryptography is surprising: the people can build a shared secret in public, because the operation that makes the secret is easy in one direction and very hard to reverse. Diffie and Hellman showed this for exponentiation in a finite group [1], and Rivest, Shamir and Adleman showed a second way, in which a public key encrypts and only a private key decrypts [2].

Both ideas rest on topics of a course in discrete mathematics. This micro-project connects them. The aim is to see a topic of the course working inside one program, so that a number such as the key 85 can be followed from a group, through an algorithm, to a message that arrives on the other side of a network.

1.2 Objectives

1. Model users, messages, keys and the cipher with sets, relations and functions, and test the properties of these objects (reflexive, symmetric, transitive, one-to-one, onto).
2. Model the communication network as a graph and analyse it: adjacency matrix and list, connectivity, shortest paths, cycles, bridge channels and cut users.
3. Implement modular arithmetic from first principles: fast exponentiation, Euclid's algorithm, the extended algorithm and the modular inverse.
4. Study the group $Z_{97}^{\times}$: check the group rules, find orders, generators and cyclic subgroups.
5. Implement the Diffie–Hellman key exchange, use the shared key in a cipher, and run a secure network in which every channel has its own key.
6. Implement RSA as a second public-key method and compare it with Diffie–Hellman.
7. Test and verify the whole system, and simulate eavesdroppers to show what the small numbers do not protect.

1.3 Scope and what is not claimed

All numbers are small (the prime is 97, the RSA modulus is 3233) so that every step can be followed and checked by hand. The cipher $E_k(m) = k \cdot m \bmod 97$ has only 96 keys and is a learning model. RSA is used in its plain "textbook" form without padding, on purpose, so that its weaknesses can be shown. The project does not protect real data and must not be used for that (Section 6).

1.4 Organisation of the report

Section 2 gives the mathematics. Section 3 describes the design of the system and Section 4 its implementation. Section 5 presents the results, the tests and the verification. Section 6 discusses security and the limits of the model, and Section 7 concludes. Appendix A explains how to run the project and Appendix B lists the code.

2 Mathematical background

2.1 Sets, relations and functions

The model starts with four sets. The set of users is $U = \{\text{Alice}, \text{Bob}, \text{Carol}, \text{Dave}, \text{Frank}, \text{Grace}\}$. The alphabet is the 95 printable ASCII characters (space to ~); a character with code c is the plaintext symbol $c - 31$, so the set of plaintext symbols is $P = \{1, \ldots, 95\}$. The set of keys is $K = \{1, \ldots, 96\}$ and the set of ciphertext values is $C = \{1, \ldots, 96\}$. The value 96 belongs to C but is not a plaintext symbol, which is how a wrong key can be noticed (Section 4.2).

A relation $R \subseteq A \times A$ is **reflexive** if $(a, a) \in R$ for all a, **symmetric** if $(a, b) \in R \Rightarrow (b, a) \in R$, **antisymmetric** if $(a, b), (b, a) \in R \Rightarrow a = b$ and **transitive** if $(a, b), (b, c) \in R \Rightarrow (a, c) \in R$. Two relations are used. R_1, "can talk with", holds for the six channels in both directions; it is symmetric, not reflexive and not transitive, so it is not an equivalence relation (Alice talks with Bob and Bob with Dave, but Alice has no channel to Dave). R_2, "owns", pairs each user with one private key.

A function $f : A \to B$ is **one-to-one** if different inputs give different outputs and **onto** if every element of B is an output. For a key $k \in K$ the encryption $E_k(m) = k \cdot m \bmod 97$ is a function from P to C. Because 97 is prime, multiplying by k permutes the non-zero residues, so E_k is one-to-one; it is not onto C, because C has 96 values and P only 95. Decryption $D_k(c) = j \cdot c \bmod 97$, with j the inverse of k modulo 97, undoes it: $D_k(E_k(m)) = m$.

2.2 Graph theory

A simple undirected graph $G = (V, E)$ has a set of vertices and a set of edges, each edge a pair of different vertices. Here the vertices are the users and the edges are the channels. The graph is stored as an **adjacency matrix** A with $A_{ij} = 1$ when i and j share a channel, and as an adjacency list. The **degree** of a vertex is the number of its edges, and the handshake lemma says that

$$\sum_{v \in V} \deg(v) = 2|E|.$$

A **path** is a sequence of vertices joined by edges, and the **distance** between two vertices is the number of edges of a shortest path (infinite when there is none). Breadth-first search (BFS) finds shortest paths: it visits the vertices in layers of increasing distance, so the first time it reaches a vertex is by a shortest path. The vertices of one connected piece form a **connected component** (a "group of users" in the program). A **cycle** is a closed path, and the number of independent cycles is

$$\text{cycle rank} = |E| - |V| + c,$$

where c is the number of components. A **bridge** is an edge whose removal increases the number of components, and a **cut vertex** is a vertex with the same property. In a

communication network they are the weak points: if a bridge channel breaks, some users can no longer reach each other.

Undirected and directed graphs. A channel carries messages in both directions, and R_1 is symmetric, so an undirected graph is the right model for this network. A directed graph (with arcs $u \to v$) is the right model when traffic or trust goes one way only. The code in this project uses undirected graphs.

2.3 Modular arithmetic

We write $a \equiv b \pmod m$ when m divides $a - b$. The **greatest common divisor** is found by Euclid's algorithm, which repeats $a = q\,b + r$ and replaces (a, b) by (b, r) until $r = 0$. The **extended** algorithm goes back through the steps and produces whole numbers x, y with

$$a\,x + b\,y = gcd(a, b).$$

When $gcd(a, m) = 1$ this gives $a\,x \equiv 1 \pmod m$, so $x \bmod m$ is the **inverse** of a modulo m; if $gcd(a, m) \neq 1$ there is no inverse. For example $gcd(85, 97) = 1$ and $85 \cdot 8 \equiv 1 \pmod{97}$, so the inverse of 85 is 8.

Fast exponentiation. To compute $b^e \bmod m$ the exponent is written in binary and processed from the left: for each bit the running value is squared, and it is multiplied by b when the bit is 1, always reducing mod m. For $5^{13} \bmod 97$ the binary form is 1101 and four squarings give 29, where the plain way needs 12 multiplications. For $5^{96} \bmod 97$ there are 7 squarings and 2 multiplications instead of 95 multiplications. This makes the large powers of Diffie–Hellman and RSA practical.

2.4 Group theory

A **group** is a set with an operation that is closed, associative, has an identity and gives every element an inverse. For a prime p the numbers $1, \ldots, p - 1$ with multiplication mod p form the group $Z_p^{\cdot}$; for $p = 97$ it has 96 elements and is commutative. The **order** of a is the smallest $k \geq 1$ with $a^k \equiv 1$. **Lagrange's theorem** says that the order of an element divides the size of the group, so $a^{p-1} \equiv 1$ (Fermat's little theorem). The order of a is found by starting at $p - 1 = 96 = 2^5 \cdot 3$ and dividing out each prime factor q while $a^{k/q} \equiv 1$ still holds.

An element g is a **generator** when its order is $p - 1$, which holds exactly when $g^{(p-1)/q} \not\equiv 1$ for every prime factor q of $p - 1$. Then the powers of g run through every element of the group, and the group is **cyclic**. For $p = 97$ the number 5 is a generator, and there are $\varphi(96) = 32$ generators in all. Every element generates a cyclic subgroup whose size is its order.

2.5 The Diffie–Hellman key exchange

Everybody knows a prime p and a generator g. Alice chooses a secret a and sends $A = g^a \bmod p$; Bob chooses b and sends $B = g^b \bmod p$. Alice computes B^a and Bob computes A^b, and

$$B^a = (g^b)^a = g^{ab} = (g^a)^b = A^b \pmod p.$$

Both now hold the key $K = g^{ab} \bmod p$. An eavesdropper sees p, g, A and B but not a or b. To find K she must find a from $A = g^a$, the **discrete logarithm** problem, for which no fast method is known in large groups. In this project $p = 97$ and $g = 5$, and the key is a number from 1 to 96, so it is a key of the cipher.

2.6 The RSA cryptosystem

Choose two different primes p and q, and put $n = pq$ and $\varphi(n) = (p-1)(q-1)$. Choose e with $gcd(e, \varphi(n)) = 1$ and let d be its inverse modulo $\varphi(n)$. The **public key** is (n, e) and the **private key** is d; the numbers p, q and $\varphi(n)$ must also stay secret. Encryption and decryption are

$$c = m^e \bmod n, \; m = c^d \bmod n.$$

The reason is the group Z_n^*, which has $\varphi(n)$ elements. By Lagrange's theorem $a^{\varphi(n)} \equiv 1$ for every a in the group. Since $ed = 1 + k\varphi(n)$, we get $(m^e)^d = m \cdot (m^{\varphi(n)})^k \equiv m$. (The same result holds for the few numbers that share a factor with n.) The security rests on the factoring problem: finding $\varphi(n)$ from n is as hard as finding p and q.

3 System design

3.1 The users and the network

There are six users. The relation R_1 gives six channels, numbered e_1 to e_6 in a fixed order, and the network is connected.

Table 1. The channels of the network and the key of each channel ($p = 97$, $g = 5$).

Channel	Users	Key (from Diffie–Hellman)
e_1	Alice – Bob	85
e_2	Alice – Carol	12
e_3	Bob – Dave	42
e_4	Carol – Dave	90
e_5	Dave – Frank	32
e_6	Frank – Grace	72

The degrees are 2, 2, 2, 3, 2 and 1 (Alice to Grace), which add up to 12 = 2 × 6, as the handshake lemma requires. The cycle rank is $6 - 6 + 1 = 1$: the only independent cycle is Alice – Bob – Dave – Carol – Alice. The bridge channels are e_5 (Dave – Frank) and e_6 (Frank – Grace), and the cut users are Dave and Frank. The shortest route from Alice to Grace is Alice – Bob – Dave – Frank – Grace, with 4 channels.

3.2 Architecture of the modules

The program has seven modules in `src/`, one folder each. They are built in the order in which the mathematics depends on itself: sets and cipher first, then the graph, then the group theory, then Diffie–Hellman and encryption, and then the integration layer that joins them. RSA is a separate module that reuses the modular arithmetic.

Table 2. The modules of the code (116 files, 4,647 lines in all; the tests add 20 files and 3,654 lines).

Module (folder in `src/`)	Models	Files	Lines
`sets_relations_functions`	Users, symbols, keys; R_1, R_2; E_k and D_k	17	434
`graph_network`	Graph, adjacency matrix and list, BFS, paths, cycles, bridges, pictures	31	931
`group_theory`	Fast power, gcd, inverse, primes, $Z_{97}^{¿}$, orders, generators	17	631
`diffie_hellman`	Public values, shared keys, key table, eavesdropper on the key exchange	9	225
`encryption`	Messages with the shared key, eavesdropper on the cipher	5	283
`integration`	Keys on channels, routing, typed demo, route picture, verification	17	994
`rsa`	RSA keys, text, users, key transport, eavesdroppers, verification	20	1,149

Flowchart 5: how the modules of the project fit together

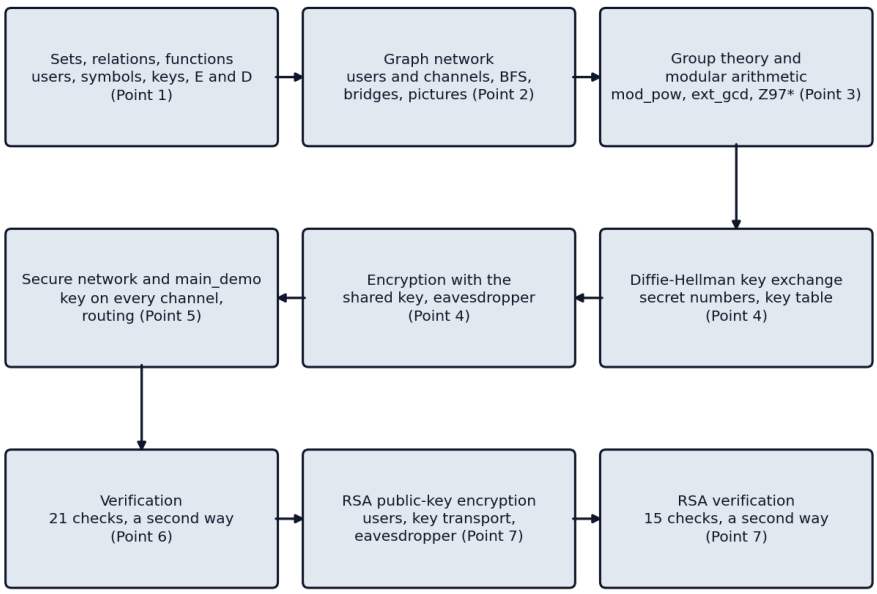


Figure 1. The architecture of the modules and how data moves between them.

4 Implementation

4.1 Code organisation and style

Every function is in its own file, and the file name is the function name. Functions named `show_*` print a whole part of the result with explanations and a list of self-checks, and each point of the project has a `demo_point<N>` command that runs these. The code uses only base MATLAB (no toolboxes) and also runs in GNU Octave, with plain ASCII in the files. Pictures are made with base `line`, `fill` and `text` commands. One rule needed care: in MATLAB both operands of `&&` and `||` must be single true-or-false values, which GNU Octave does not insist on. The first run in MATLAB (R2026b) found one place where `all()` of a matrix was used as an operand; it was fixed with `all(all(...))`.

4.2 The cipher with the key (module `sets_relations_functions`)

`encrypt_symbols` computes $c = k \cdot m \bmod 97$ and `decrypt_symbols` computes $m = j \cdot c \bmod 97$ with j from `key_inverse`. `text_to_symbols` and `symbols_to_text` convert between text and symbols. If a decryption gives the value 96 (which is in C but is not a symbol), `decrypt_symbols` stops with an error. This is what happens with a wrong key, and it lets the program notice a wrong key at once. `relation_properties` and `function_properties` test the properties of Section 2.1 from the 0/1 matrix of a relation.

4.3 The graph (module `graph_network`)

`define_graph` builds the graph from R_1 and stores each edge once as $[i\ j]$ with $i < j$. `bfs_search` visits the neighbours of a vertex in increasing order, so the shortest path shown is always the same one. `shortest_path`, `distance_matrix`, `connected_components`, `cycle_rank` and `find_cycle` follow the definitions of Section 2.2. `bridge_channels` and `cut_users` remove each channel or user in turn and count the components. `graph_layout` fixes the place of every user (the users are drawn in layers by distance from the first user of a group), so a picture of the same network always looks the same, and `draw_network` draws the picture. Five pictures are made: the network, the shortest path, a shortest cycle, the weak points (bridges red, cut users orange) and the network after the first bridge is removed.

4.4 Modular arithmetic and the group (module `group_theory`)

`mod_pow` is the repeated squaring of Section 2.3. It needs the modulus to be at most 2^{26} so that every product stays below 2^{52} and the double-precision numbers stay exact. `euclid_gcd` gives the steps $[a\ b\ q\ r]$, `ext_gcd` gives g, x, y with $a x + b y = g$, and `mod_inverse` uses it and stops with an error when $gcd(a, m) \neq 1$. `check_group` takes any multiplication table and tests closure, identity, inverses, associativity (all triples) and commutativity; applied to Z_{97}^{*} all five rules hold. `element_order`, `is_generator`, `find_generators` and `cyclic_subgroup` follow Section 2.4. The tests compare the fast method for orders with the plain method of multiplying until the result is 1.

4.5 Diffie–Hellman and encryption (modules `diffie_hellman`, `encryption`)

`dh_public_value` computes $g^a \bmod p$ and `dh_shared_key` computes $B^a \bmod p$. `dh_exchange` runs one exchange and checks that both keys agree and equal $g^{ab \bmod (p-1)} \bmod p$. `check_dh_setup` refuses a p that is not prime and a g that is not a generator. With the default secret numbers (Alice 6, Bob 15, Carol 23, Dave 41, Frank 58, Grace 77) the public values are 8, 46, 82, 80, 44 and 23. `send_message` lets a sender and a receiver each compute the key from their own secret and the public value of the other, encrypt with one key and decrypt with the other, and report whether the text came back.

4.6 The secure network and the routing (module `integration`)

`define_system` joins everything in one structure: the users, the graph, the secrets and the key of every channel (the keys of Table 1). `route_message` sends a text along the shortest path of the working network, hop by hop; every hop is a `send_message` with the key of that channel, so a relay decrypts what it gets and encrypts it again with the key of the next channel. If there is no route, the message is not sent, and this is not treated as an error. `main_demo` is the entry command for the demo: it prints a menu and asks for every answer with typed prompts. The user can send a message (choosing sender, receiver and text), see the key exchange of two users, see the channels and their keys, break a channel, repair the channels, print the whole report, and draw the picture of the network with the route of the last message. The function `route_picture_options` colours the picture: the route is thick and green, broken channels are dotted and grey, and when the network is split every group of users has its own colour.

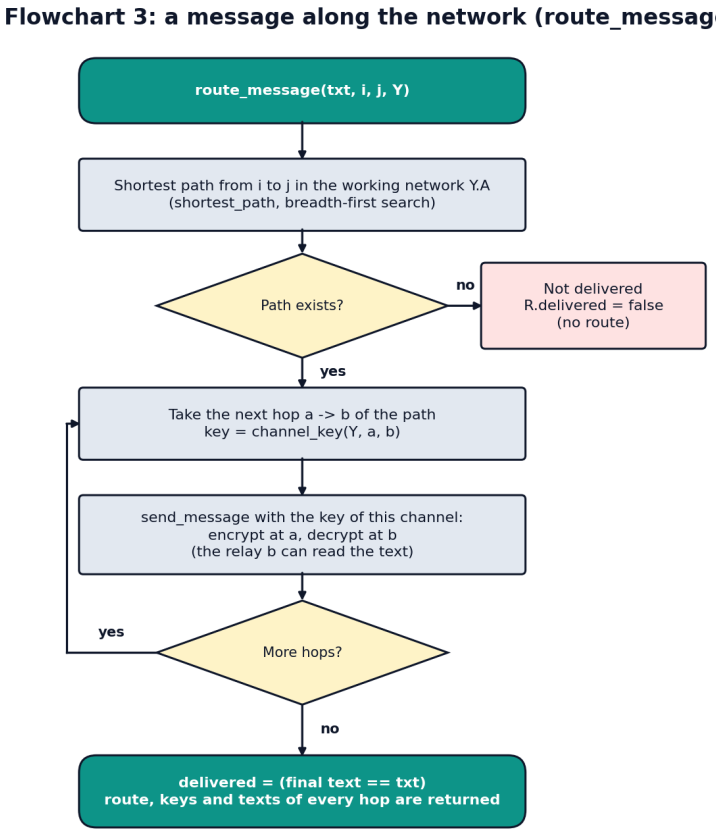


Figure 2. Routing a message along the network: every hop uses the key of its own channel.

4.7 The eavesdroppers

Three attackers are simulated. `eavesdrop_dh` sees p, g, A and B and computes $g^1, g^2, \ldots$ until she reaches A, which gives the secret a and then the key B^a. `try_all_keys` decrypts a ciphertext with every one of the 96 keys and keeps those that give readable text. In RSA, `eavesdrop_rsa` factors n by trial division and builds $\varphi(n)$ and d, and `eavesdrop_codebook` encrypts every number from 1 to 96 with the public key and looks the message up in that list. All of them succeed on the small numbers of the project, which is the purpose of the simulation.

4.8 RSA (module `rsa`)

`check_rsa_setup` and `rsa_make_keys` implement Section 2.6: they check that p and q are different primes, that n is larger than 96 (so that every symbol and every key is smaller than n) and at most 2^{26}, and that $gcd(e, \varphi(n)) = 1$, and then they find d with the extended Euclid algorithm. `rsa_encrypt_number`, `rsa_decrypt_number`, `rsa_encrypt_text` and `rsa_decrypt_text` use `mod_pow`. Text is encrypted one symbol at a time (textbook RSA). `define_rsa_users` gives each of the six users their own key pair, and `rsa_send_message` encrypts with the public key of the receiver only, so the sender needs no secret.

Key transport. RSA needs a modular power for every number, which is slow for a long text. `rsa_key_transport` therefore sends only the cipher key k with RSA, as $k^e \bmod n$, and

encrypts the text with the fast cipher E_k. For Bob (with $d = 157$) one RSA decryption takes 8 squarings and 5 multiplications, 13 modular multiplications. For the 36-character demo text, RSA on every number needs $36 \times 13 = 468$ multiplications, while key transport needs $13 + 36 = 49$.

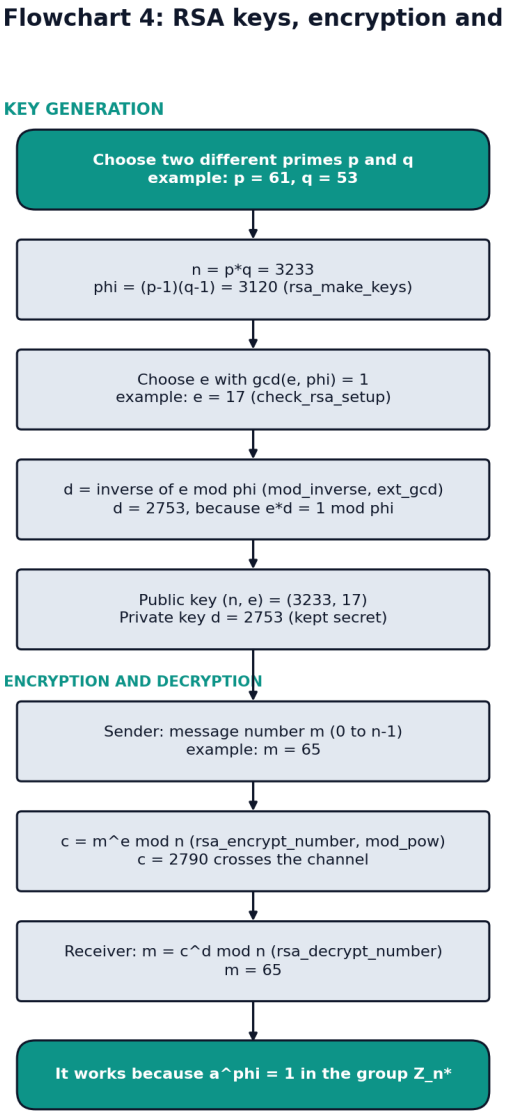


Figure 3. RSA: making the keys, encrypting and decrypting, and the key transport.

5 Results and verification

5.1 The test suite

Every part has a test file of automatic checks that print PASS or FAIL lines. All 14 test files that need no figure window were run in GNU Octave 8.4 and every check passed.

Table 3. Results of the test suite in GNU Octave 8.4.

Test file	Part tested	Checks passed
`test_sets`	Sets, text conversion	25 of 25
`test_relations`	Relations and their properties	51 of 51
`test_cipher`	Encryption and decryption as functions	50 of 50
`test_graph`	Graph, adjacency matrix and list	51 of 51
`test_modular`	Fast power, gcd, inverse	130 of 130
`test_group`	Group rules, orders, generators	131 of 131
`test_dh`	Diffie–Hellman	71 of 71
`test_encryption`	Encryption with the shared key, eavesdroppers	101 of 101
`test_system`	Secure network and routing	69 of 69
`test_verification`	Verification of key, decryption and graph	32 of 32
`test_rsa`	RSA keys, numbers and text	109 of 109
`test_rsa_users`	RSA users, key transport, eavesdroppers	188 of 188
`test_rsa_verification`	Verification of RSA	27 of 27
`test_main_demo`	The typed demo (without the keyboard)	59 of 59
Total		**1,094 of 1,094**

Three more test files (`test_graph_analysis`, `test_graph_pictures` and `test_route_picture`) draw figures and are run in MATLAB by the team. The tests never write into the `outputs` folder: the pictures are drawn with the figure hidden and saved in a temporary folder. The tests also check that bad input is refused, for example a prime p that is not prime, a generator that is not a generator, a secret number out of range, or a text with a character outside the alphabet.

5.2 The Diffie–Hellman key exchange

With $p = 97$ and $g = 5$, Alice chooses $a = 6$ and sends $A = 5^{6} \bmod 97 = 8$. Bob chooses $b = 15$ and sends $B = 5^{15} \bmod 97 = 46$. Alice computes $46^{6} \bmod 97 = 85$ and Bob computes $8^{15} \bmod 97 = 85$. Both get the key 85, which agrees with $g^{ab} = 5^{90} \bmod 97 = 85$; only 8 and 46 crossed the channel. Table 4 gives the key of every pair of users. The table is symmetric, so two users always get the same key.

Table 4. The shared Diffie–Hellman key of every pair of users.

	Alice	Bob	Carol	Dave	Frank	Grace
Alice	–	85	12	89	33	18
Bob	85	–	67	42	8	28
Carol	12	67	–	90	11	60
Dave	89	42	90	–	32	41
Frank	33	8	11	32	–	72
Grace	18	28	60	41	72	–

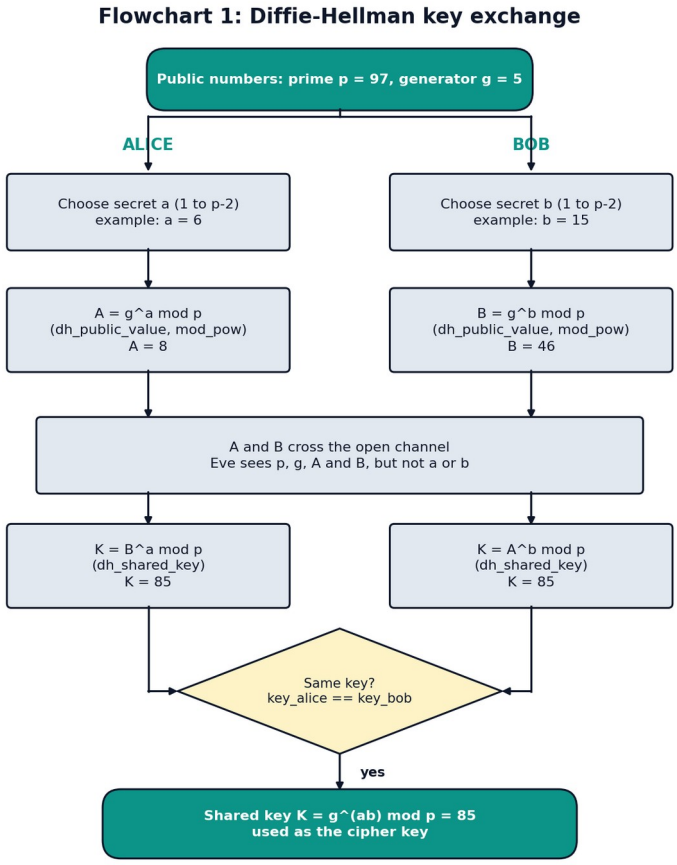


Figure 4. The Diffie–Hellman key exchange step by step.

5.3 Encryption with the shared key

Alice sends Bob the text *Meet me at the library at 5 pm, Bob.* (36 characters). The first character M is the symbol 46, and it becomes $85 \cdot 46 \bmod 97 = 30$. The 36 numbers that cross the channel start 30, 33, 33, 47, 85, 34. Bob finds the inverse key $j = 8$ (because $85 \cdot 8 \bmod 97 = 1$) and gets the same text back. If Bob uses the key of Alice and Carol (12) instead, the decryption stops, because a value outside the symbols comes out. All 30 ordered pairs of users send and receive the text correctly.

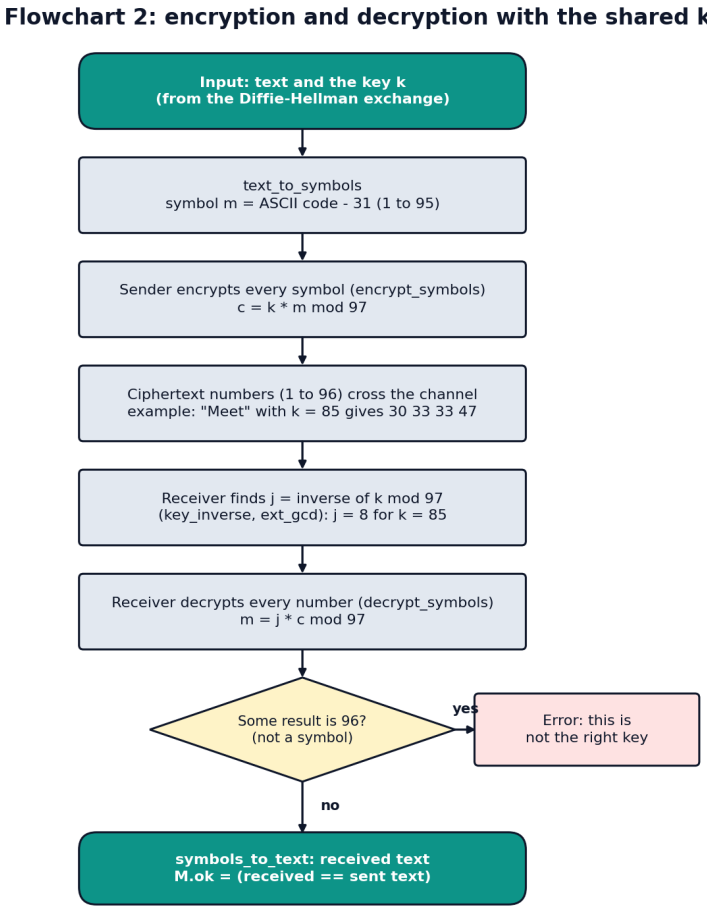


Figure 5. Encryption and decryption with the shared key.

5.4 The network at work

A message from Alice to Grace, *Hello Grace, the meeting is at 5 pm.*, goes along Alice – Bob – Dave – Frank – Grace. The four hops use the keys 85, 42, 32 and 72, and on every hop the numbers are different, because the keys are different (the first hop starts 90, 33, 46, 46, the second 73, 30, 33, 33). Grace gets the text intact, and Carol, who is not on the route, cannot read it. The users in the middle can read the message, because each of them decrypts it before encrypting it again: the encryption protects each channel, not the whole route. When all 30 ordered pairs send a message, the number of messages by route length is 12 (1 channel), 10 (2 channels), 6 (3 channels) and 2 (4 channels), and all 30 arrive.

When the channel Dave – Frank (a bridge) is removed, the network falls apart into the groups {Alice, Bob, Carol, Dave} and {Frank, Grace}. The message from Alice to Grace then has no route and is not sent, and only 14 of the 30 messages still arrive.

5.5 Verification by a second method

A test that uses the same code as the program can repeat the program’s mistake. The verification layer therefore checks each important result a second way, with code that shares

nothing with the part it checks. `verify_system` has 21 checks and `verify_rsa` has 15, and they are listed in Tables 5 and 6.

Table 5. The 21 checks of `verify_system`. The same 21 checks also agree on the network after the channel Dave – Frank is removed.

Group	Check (the second method)	Cases	Agree
Shared key	Both users of a pair compute the same key	15	yes
	The key is $g^{ab} \bmod 97$ by plain repeated multiplication	15	yes
	Every key is a number from 1 to 96	15	yes
	The key table is symmetric, with 0 on the diagonal	15	yes
	Every channel has the key of its two users	6	yes
	An eavesdropper who tries every secret finds the same key	15	yes
Cipher	A text with all 95 characters arrives for every pair	30	yes
	$k \cdot m$ and $j \cdot c$ worked out by hand (j found by trial)	15	yes
	Every wrong key fails to give the text back (15 keys × 95 wrong keys)	1,425	yes
	A message arrives exactly when there is a route	30	yes
	Every hop uses the key of its channel	30	yes
	Values on a channel are in the ciphertext set (1 to 96)	30	yes
Graph	The matrix is symmetric and has no loops	36	yes
	The degrees add up to twice the number of channels	6	yes
	The adjacency list gives back the matrix	6	yes
	Distances are finite exactly where matrix products show a path	36	yes
	Every distance is the smallest power k of the matrix with a path	36	yes
	Every shortest path uses only channels and has the right length	30	yes
	The groups of users equal the groups found with matrix products	6	yes
	A channel is a bridge exactly when removing it makes more groups	6	yes
	A user is a cut user exactly when removing him makes more groups	6	yes

The second methods are: repeated multiplication instead of `mod_pow`; the formulas $k \cdot m \bmod p$ and $j \cdot c \bmod p$ instead of `encrypt_symbols` and `decrypt_symbols`; powers of the adjacency matrix instead of breadth-first search (a walk of k channels from i to j exists exactly when entry (i, j) of the k-th power of the matrix is not zero, so the smallest such k is the distance); and removing a channel or a user and counting the groups instead of `bridge_channels` and `cut_users`. The test for the verification also checks that the verification *finds* mistakes: a wrong channel key and a wrong public value are both caught.

For RSA, `verify_rsa` makes 15 checks in the same three groups (the RSA keys, encryption and decryption, and the key transport with the eavesdropper), with trial division instead of `is_prime_number`, trying every number instead of `mod_inverse`, repeated multiplication instead of `mod_pow`, and counting the numbers with no common divisor with n instead of the formula for $\varphi(n)$. One finding is worth noting: the check on d tries every d from 1 to $\varphi(n)-1$ and shows that a d works for every number from 0 to $n-1$ exactly when $e\,d \equiv 1 \pmod{lcm(p-1, q-1)}$, so a user can have more than one working private key (Alice: 413, 1193, 1973 and 2753). The checks are repeated for a second set of keys.

5.6 RSA results

For $p=61$, $q=53$, $e=17$ the modulus is $n=3233$ and $\varphi(n)=3120$. The extended Euclid algorithm gives $17\cdot(-367)+3120\cdot 2=1$, so $d=-367 \bmod 3120=2753$. The message $m=65$ gives $c=65^{17} \bmod 3233=2790$, and $2790^{2753} \bmod 3233=65$. Counting the numbers with no common factor with 3233 gives exactly 3120, and $a^{3120}\equiv 1$ for all of them. Also $e\,d=46801=1+15\cdot 3120$, as the proof in Section 2.6 requires.

The same demo text encrypts to 36 numbers from 0 to 3232, the first being $46^{17} \bmod 3233=2825$. The 8 spaces all become the number 1, because the space is the symbol 1 and $1^{e}=1$: equal characters give equal numbers, which is the weakness of textbook RSA. The six users have the keys of Table 7.

Table 7. The RSA keys of the six users (only n and e are public).

User	p	q	n	e	d
Alice	61	53	3233	17	2753
Bob	47	59	2773	17	157
Carol	43	67	2881	5	1109
Dave	71	37	2627	11	2291
Frank	79	31	2449	7	1003
Grace	73	41	2993	7	823

5.7 Figures of the network

Six pictures show the network of the six users. Figures 6 to 10 are made by `demo_point2` and Figure 11 by option 7 of `main_demo`. In every picture the users keep the places that `graph_layout` gives for the whole network, so only the colors, the line widths, the line styles and the title change from one picture to the next. The channels are numbered e1 to e6 in the order Alice–Bob, Alice–Carol, Bob–Dave, Carol–Dave, Dave–Frank and Frank–Grace.

Note on these figures. The six pictures in this version of the report were not copied from MATLAB windows. A separate plotting script (`handoff/make_figures.py`) drew them from the places, colors, line widths and titles that the MATLAB functions `graph_layout`, `picture_options` and `route_picture_options` compute (these values were read out with GNU Octave, which cannot open figure windows). They have the layout and the colors of

`draw_network`, and they will be replaced by the PNG files that `demo_point2` and `main_demo` save in MATLAB (same file names) after the run on the demo laptop.

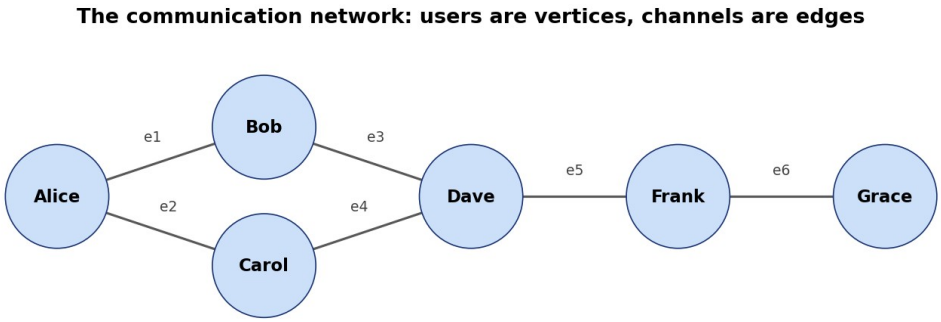


Figure 6. The communication network: the users are the vertices and the six channels are the edges (`point2_network.png`).

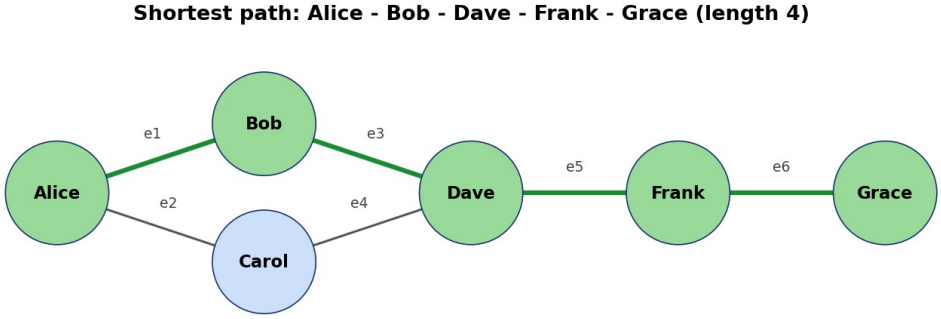


Figure 7. The shortest path from Alice to Grace: Alice – Bob – Dave – Frank – Grace, 4 channels (`point2_shortest_path.png`).

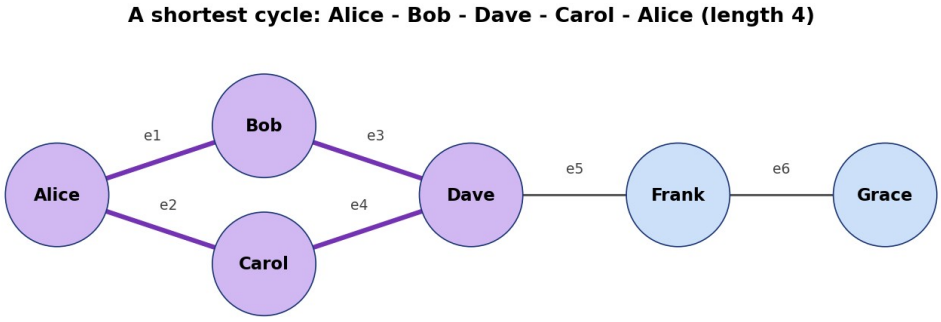


Figure 8. A shortest cycle: Alice – Bob – Dave – Carol – Alice, length 4. The network has exactly one independent cycle, since 6 channels − 6 users + 1 group = 1 (`point2_cycle.png`).

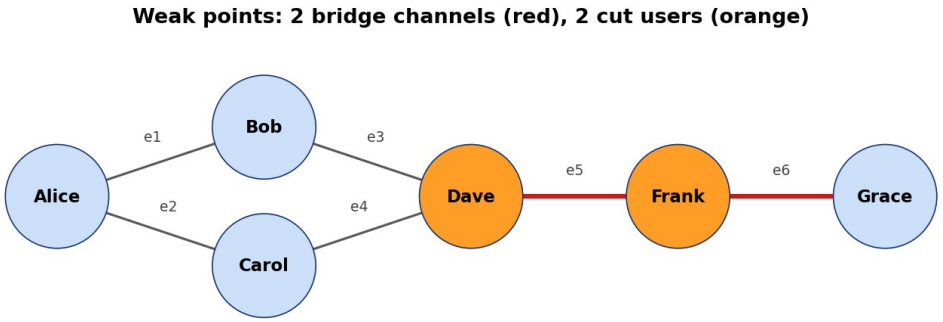


Figure 9. The weak points: the bridge channels Dave – Frank (e5) and Frank – Grace (e6) are red, and the cut users Dave and Frank are orange (`point2_weak_points.png`).

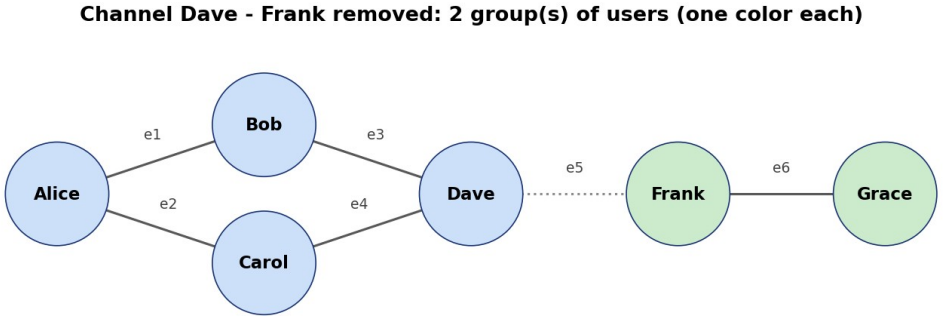


Figure 10. The network after the bridge channel Dave – Frank is removed (dotted): two groups of users, {Alice, Bob, Carol, Dave} and {Frank, Grace}, one color each (`point2_split.png`).

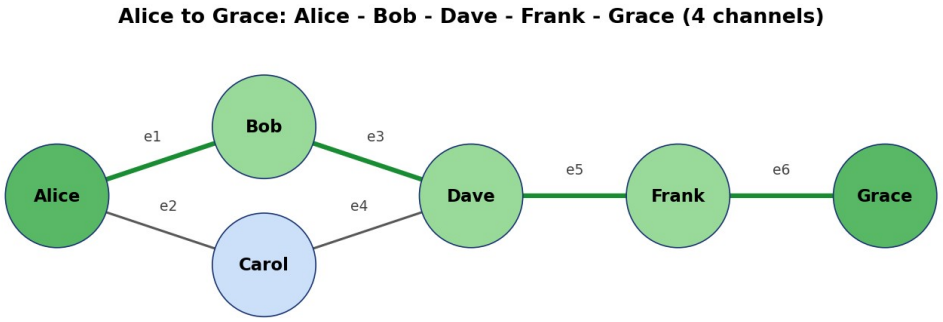


Figure 11. The route of the message from Alice to Grace in `main_demo` (option 7). The thick green channels e1, e3, e5 and e6 carry it with the keys 85, 42, 32 and 72; Alice and Grace are dark green, Bob, Dave and Frank light green, and Carol is not on the route (`point5_route.png`).

6 Security analysis and limitations

What the simulation shows. The eavesdropper `eavesdrop_dh` finds the secret number and the key of Alice and Bob from $A = 8$ by trying $g^1, g^2, \ldots$, which takes only a few steps because the group has 96 elements. For RSA, trial division finds the primes of n with at most $\sqrt{n}$ tries, and Table 8 shows how the work grows with the size of n.

Table 8. Eve factors n by trial division: the number of tries grows with $\sqrt{n}$.

n	Bits	Tries of Eve	Most tries	Eve gets the right d
143	8	10	10	yes
10,403	14	100	100	yes
1,022,117	20	1,008	1,009	yes
64,160,099	26	8,008	8,008	yes

The number of tries grows with the square root of n. A modulus of 2048 bits (a number with 617 digits) would need up to 2^{1024} tries by this method, which is impossible. Faster factoring methods exist, but they also need a very large amount of work for such a modulus, which is why real systems use numbers of that size.

Why the cipher is weak. The cipher $E_k(m) = k \cdot m \bmod 97$ has only 96 keys, so Eve can try them all: `try_all_keys` shows that for the demo message only the key 85 gives readable text. Because the cipher is linear, a single known pair of plaintext and ciphertext also reveals k. The model is therefore good for learning how a shared key is used, and not for protection.

Other limits.

1. *Not end-to-end.* In the routing, every relay decrypts and re-encrypts, so the users in the middle read the message. A real system would encrypt end to end.
2. *No authentication.* A Diffie–Hellman exchange without signatures or certificates can be attacked by a person in the middle who runs two exchanges. This is not simulated.
3. *Textbook RSA.* The encryption has no padding, so equal characters give equal numbers, and the list of all small numbers (`eavesdrop_codebook`) reads the message without factoring n.
4. *Small numbers.* The prime is limited to 2^{26} so that `mod_pow` stays exact in double-precision arithmetic.
5. *Undirected graph.* All channels are two-way (Section 2.2).

7 Conclusion and future work

The project connects four topics of discrete mathematics in one working program. Sets, relations and functions describe the users and the cipher. Graph theory finds the route of a message and the weak points of the network. Group theory explains why the Diffie–Hellman key is the same on both sides, and why RSA decrypts correctly. Number theory supplies the algorithms that make it fast. The result is a 116-file MATLAB program with 1,094 automatic checks that pass in GNU Octave, and a verification layer that checks the key, the decryption, the graph and RSA a second way. The simulated attackers show that the small numbers give no protection, which is the main lesson of the model.

Possible future work: use a modulus with big integers (so that real sizes can be shown), add padding to RSA, authenticate the Diffie–Hellman exchange with signatures, encrypt end to end instead of hop by hop, use elliptic-curve groups instead of $Z_p^{\times}$, and add directed channels to the graph model.

References

[1] W. Diffie and M. E. Hellman, “New directions in cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.

[2] R. L. Rivest, A. Shamir and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[3] D. R. Stinson, *Cryptography: Theory and Practice*, CRC Press.

[4] K. H. Rosen, *Discrete Mathematics and Its Applications*, McGraw-Hill.

[5] D. B. West, *Introduction to Graph Theory*, Prentice Hall.

[6] J. A. Gallian, *Contemporary Abstract Algebra*, Cengage Learning.

[7] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, Addison-Wesley.

Appendix A: How to run the project

1. Open MATLAB and set the current folder to the project folder.
2. Run `setup_project` to add the code folders to the MATLAB path.
3. Run `run_all_tests`; it ends with ALL TESTS PASSED when everything is correct.
4. Run the demos: `demo_point1` (sets, relations, functions), `demo_point2` (graph, analysis and 5 pictures, saved in `outputs`), `demo_point3` (modular arithmetic and the group), `demo_point4` (Diffie–Hellman, encryption and the eavesdropper), `demo_point5` (the secure network), `demo_point6` (the verification) and `demo_point7` (RSA).
5. Run `main_demo` for the demo with typed prompts. Choose option 1 to send a message, option 4 to break a channel, option 5 to repair, option 7 to draw the picture of the route (saved as `outputs/point5_route.png`), and option 0 to quit.

The README of the project folder lists the same commands, and `PROGRESS.md` records every design decision. The seminar slides are in `docs/Crypto_Microproject_Presentation.pptx` and the flowcharts in `docs/flowcharts`.

Appendix B: The code, file by file

Each function is one file in `src/<module>/`, and the file name is the function name. The `show_*` functions print a whole part of the results with explanations and self-checks, and the `demo_point*` commands run them.

Table 9. The functions of each module (116 files in `src/`).

Module	Functions	Printouts and demos
Sets, relations and functions	`cipher_relation`, `decrypt_symbols`, `decrypt_text`, `define_relations`, `define_sets`, `encrypt_symbols`, `encrypt_text`, `function_properties`, `key_inverse`, `pairs_to_matrix`, `relation_properties`, `symbols_to_text`, `text_to_symbols`	`demo_point1`, `show_cipher`, `show_relations`, `show_sets`
Graph network	`adjacency_to_edges`, `adjacency_to_list`, `bfs_search`, `bridge_channels`, `check_picture_files`, `connected_components`, `cut_users`, `cycle_rank`, `define_graph`, `distance_matrix`, `draw_network`, `edges_to_adjacency`, `find_cycle`, `format_groups`, `format_names`, `graph_layout`, `is_connected`, `list_to_adjacency`, `mark_edges`, `path_to_edges`, `picture_options`, `plural_s`, `reachable_from`, `remove_channel`, `remove_user`, `save_network_pictures`, `shortest_path`, `yes_no`	`demo_point2`, `show_analysis`, `show_graph`
Group theory and modular arithmetic	`check_group`, `cyclic_subgroup`, `element_order`, `euclid_gcd`, `ext_gcd`, `find_generators`, `group_table`, `is_generator`, `is_prime_number`, `is_whole_number`, `mod_inverse`, `mod_pow`, `prime_factors`	`demo_point3`, `show_cyclic`, `show_group`, `show_modular`
Diffie-Hellman	`check_dh_setup`, `define_secrets`, `dh_exchange`, `dh_key_table`, `dh_public_value`, `dh_shared_key`, `eavesdrop_dh`	`demo_point4`, `show_dh`
Encryption	`format_numbers`, `send_message`, `try_all_keys`	`show_eavesdropper`, `show_encryption`
Integration, demo and verification	`channel_key`, `check_system`, `define_system`, `draw_route_picture`, `main_demo`, `pick_user`, `reach_by_matrix`, `read_answer`, `route_message`, `route_picture_options`, `verify_system`	`demo_point5`, `demo_point6`, `show_route`, `show_route_picture`, `show_system`, `show_verification`

Module	Functions	Printouts and demos
RSA	check_rsa_setup, check_rsa_users, define_rsa_users, eavesdrop_codebook, eavesdrop_rsa, rsa_decrypt_number, rsa_decrypt_text, rsa_encrypt_number, rsa_encrypt_text, rsa_format_numbers, rsa_key_transport, rsa_make_keys, rsa_send_message, verify_rsa	demo_point7, show_rsa, show_rsa_eavesdropper, show_rsa_transport, show_rsa_users, show_rsa_verification